Assignment 16.4

N SOURISH

2303A52360

Batch: 44

TASK1:

Prompt: Generate SQL CREATE TABLE statements for Instructors, Courses, and Students with primary keys, foreign keys, and proper data types. Keep the schema normalized.

```
# ----------------------------------------------------------------
# Task 1: Database Schema Design
# ----------------------------------------------------------------
print_heading("TASK 1: DATABASE SCHEMA DESIGN")

create_instructors = """
CREATE TABLE IF NOT EXISTS Instructors (
    instructor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    country TEXT NOT NULL
);
"""

create_courses = """
CREATE TABLE IF NOT EXISTS Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    category TEXT NOT NULL,
    price REAL NOT NULL CHECK (price >= 0),
    start_date TEXT,
    instructor_id INTEGER NOT NULL,
    FOREIGN KEY (instructor_id)
        REFERENCES Instructors(instructor_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);
"""

create_students = """
CREATE TABLE IF NOT EXISTS Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    enrollment_date TEXT NOT NULL
);
"""

# Bridge table for normalized many-to-many relation between Students and Courses.
create_enrollments = """
CREATE TABLE IF NOT EXISTS Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    enrolled_on TEXT NOT NULL DEFAULT (DATE('now')),
    UNIQUE(student_id, course_id),
    FOREIGN KEY (student_id)
        REFERENCES Students(student_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (course_id)
        REFERENCES Courses(course_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
"""
```

```
"""

for ddl in [create_instructors, create_courses, create_students, create_enrollments]:
    cursor.execute(ddl)

conn.commit()
print("Tables created successfully: Instructors, Courses, Students, Enrollments")
```

OUTPUT:

```
================================================================================
TASK 1: DATABASE SCHEMA DESIGN
================================================================================
Tables created successfully: Instructors, Courses, Students, Enrollments
```

Observation: Tables were created successfully with valid key constraints. A separate Enrollments table was used to model many-to-many student-course relationships, improving normalization and avoiding duplicate data.

Task 2

Prompt: Generate INSERT queries for at least 3 instructors, 4 courses, and 3 students, then verify data using SELECT queries.

```python
# ---------------------------------------------------------------
# Task 2: Populate Database with Sample Records
# ---------------------------------------------------------------
print_heading("TASK 2: SAMPLE DATA INSERTION")

instructors_data = [
    ("Dr. Alice Johnson", "alice.johnson@learnhub.com", "USA"),
    ("Prof. Rohan Mehta", "rohan.mehta@learnhub.com", "India"),
    ("Ms. Elena Rossi", "elena.rossi@learnhub.com", "Italy"),
]
cursor.executemany(
    "INSERT INTO Instructors (full_name, email, country) VALUES (?, ?, ?);",
    instructors_data,
)

courses_data = [
    ("SQL Fundamentals", "Database", 49.99, "2026-04-10", 1),
    ("Python for Data Analysis", "Programming", 79.99, "2026-04-15", 2),
    ("Web Development Bootcamp", "Web", 89.00, "2026-04-20", 1),
    ("Machine Learning Basics", "AI", 99.00, "2026-04-25", 3),
]
cursor.executemany(
    """
    INSERT INTO Courses (title, category, price, start_date, instructor_id)
    VALUES (?, ?, ?, ?, ?);
    """,
    courses_data,
)

students_data = [
    ("Nina Sharma", "nina.sharma@email.com", "2026-03-01"),
    ("Arjun Patel", "arjun.patel@email.com", "2026-03-03"),
    ("Sofia Kim", "sofia.kim@email.com", "2026-03-05"),
]
cursor.executemany(
    "INSERT INTO Students (full_name, email, enrollment_date) VALUES (?, ?, ?);",
    students_data,
)

enrollments_data = [
    (1, 1, "2026-03-10"),
    (1, 2, "2026-03-10"),
    (2, 1, "2026-03-11"),
    (2, 3, "2026-03-11"),
    (3, 2, "2026-03-12"),
    (3, 4, "2026-03-12"),
]
cursor.executemany(
    "INSERT INTO Enrollments (student_id, course_id, enrolled_on) VALUES (?, ?, ?);",
    enrollments_data,
)

conn.commit()
print("Sample records inserted successfully.")

print("\nVerification with SELECT *:")
print("\nInstructors:")
```

```python
print_query_results(cursor, "SELECT * FROM Instructors;")
print("\nCourses:")
print_query_results(cursor, "SELECT * FROM Courses;")
print("\nStudents:")
print_query_results(cursor, "SELECT * FROM Students;")
```

OUTPUT:

```
================================================================================
TASK 2: SAMPLE DATA INSERTION
================================================================================
Sample records inserted successfully.

Verification with SELECT *:

Instructors:
instructor_id | full_name | email | country
--------------------------------------------------------------------------------
1 | Dr. Alice Johnson | alice.johnson@learnhub.com | USA
2 | Prof. Rohan Mehta | rohan.mehta@learnhub.com | India
3 | Ms. Elena Rossi | elena.rossi@learnhub.com | Italy

Courses:
course_id | title | category | price | start_date | instructor_id
--------------------------------------------------------------------------------
1 | SQL Fundamentals | Database | 49.99 | 2026-04-10 | 1
2 | Python for Data Analysis | Programming | 79.99 | 2026-04-15 | 2
3 | Web Development Bootcamp | Web | 89.0 | 2026-04-20 | 1
4 | Machine Learning Basics | AI | 99.0 | 2026-04-25 | 3

Students:
student_id | full_name | email | enrollment_date
--------------------------------------------------------------------------------
1 | Nina Sharma | nina.sharma@email.com | 2026-03-01
2 | Arjun Patel | arjun.patel@email.com | 2026-03-03
3 | Sofia Kim | sofia.kim@email.com | 2026-03-05
```

Observation: All sample records inserted without errors. SELECT results displayed all inserted rows correctly for Instructors, Courses, and Students.

Task 3

Prompt: Perform CRUD operations: retrieve all courses, update one student email, delete one course, and verify each change using SELECT.

```python
# ------------------------------------------------------------------
# Task 3: Core CRUD Operations
# ------------------------------------------------------------------
print_heading("TASK 3: CRUD OPERATIONS")

print("1) Retrieve all courses offered:")
print_query_results(
    cursor,
    """
    SELECT c.course_id, c.title, c.category, c.price, i.full_name AS instructor
    FROM Courses c
    JOIN Instructors i ON c.instructor_id = i.instructor_id
    ORDER BY c.course_id;
    """,
)

print("\n2) Update a student's email address (student_id = 2):")
cursor.execute(
    "UPDATE Students SET email = ? WHERE student_id = ?;",
    ("arjun.patel.new@email.com", 2),
)
conn.commit()
print_query_results(
    cursor,
    "SELECT student_id, full_name, email FROM Students WHERE student_id = 2;",
)

print("\n3) Remove a course from the database (course_id = 3):")
cursor.execute("DELETE FROM Courses WHERE course_id = ?;", (3,))
conn.commit()

print("Remaining courses after delete:")
print_query_results(cursor, "SELECT course_id, title FROM Courses ORDER BY course_id;")
print("\nEnrollments after delete (course-linked rows removed via CASCADE):")
print_query_results(
    cursor,
    "SELECT enrollment_id, student_id, course_id FROM Enrollments ORDER BY enrollment_id;",
)
```

OUTPUT:

```
================================================================================
TASK 3: CRUD OPERATIONS
================================================================================
1) Retrieve all courses offered:
course_id | title | category | price | instructor
--------------------------------------------------------------------------------
1 | SQL Fundamentals | Database | 49.99 | Dr. Alice Johnson
2 | Python for Data Analysis | Programming | 79.99 | Prof. Rohan Mehta
3 | Web Development Bootcamp | Web | 89.0 | Dr. Alice Johnson
4 | Machine Learning Basics | AI | 99.0 | Ms. Elena Rossi

2) Update a student's email address (student_id = 2):
student_id | full_name | email
--------------------------------------------------------------------------------
2 | Arjun Patel | arjun.patel.new@email.com

3) Remove a course from the database (course_id = 3):
Remaining courses after delete:
course_id | title
--------------------------------------------------------------------------------
1 | SQL Fundamentals
2 | Python for Data Analysis
4 | Machine Learning Basics

Enrollments after delete (course-linked rows removed via CASCADE):
enrollment_id | student_id | course_id
--------------------------------------------------------------------------------
1 | 1 | 1
2 | 1 | 2
3 | 2 | 1
5 | 3 | 2
6 | 3 | 4
```

Observation: Course list was retrieved correctly. Student email update reflected immediately. Selected course was deleted successfully, and related enrollment rows were removed through cascading rules with no unintended data loss.

Task 4

Prompt: Create aggregate SQL queries to count enrollments per course and filter courses with enrollments greater than a threshold using GROUP BY and HAVING.

```python
# ------------------------------------------------------------------
# Task 4: Aggregate Queries and Reporting
# ------------------------------------------------------------------
print_heading("TASK 4: AGGREGATE REPORTS")

print("1) Count students enrolled in each course:")
print_query_results(
    cursor,
    """
    SELECT c.course_id,
           c.title,
           COUNT(e.student_id) AS enrolled_students
    FROM Courses c
    LEFT JOIN Enrollments e ON c.course_id = e.course_id
    GROUP BY c.course_id, c.title
    ORDER BY c.course_id;
    """,
)

threshold = 1
print(f"\n2) Courses with more than {threshold} enrollment(s):")
print_query_results(
    cursor,
    """
    SELECT c.course_id,
           c.title,
           COUNT(e.student_id) AS enrolled_students
    FROM Courses c
    LEFT JOIN Enrollments e ON c.course_id = e.course_id
    GROUP BY c.course_id, c.title
    HAVING COUNT(e.student_id) > ?
    ORDER BY enrolled_students DESC;
    """,
    (threshold,),
)
```

OUTPUT:

```
================================================================================
TASK 4: AGGREGATE REPORTS
================================================================================
1) Count students enrolled in each course:
course_id | title | enrolled_students
--------------------------------------------------------------------------------
1 | SQL Fundamentals | 2
2 | Python for Data Analysis | 2
4 | Machine Learning Basics | 1

2) Courses with more than 1 enrollment(s):
course_id | title | enrolled_students
--------------------------------------------------------------------------------
1 | SQL Fundamentals | 2
2 | Python for Data Analysis | 2
```

Observation: Aggregate results were accurate. GROUP BY produced per-course counts, and HAVING correctly filtered courses above the given enrollment limit.

Task 5

Prompt: Optimize a query that fetches courses taught by instructors from a specific country using efficient joins and indexing suggestions; compare old and new results.

```python
    # ------------------------------------------------------------------
    # Task 5: Query Optimization with AI Suggestions
    # ------------------------------------------------------------------
    print_heading("TASK 5: QUERY OPTIMIZATION")

    target_country = "USA"

    original_query = """
    SELECT course_id, title, instructor_id
    FROM Courses
    WHERE instructor_id IN (
        SELECT instructor_id
        FROM Instructors
        WHERE country = ?
    )
    ORDER BY course_id;
    """

    optimized_query = """
    SELECT c.course_id, c.title, c.instructor_id
    FROM Courses c
    INNER JOIN Instructors i
        ON c.instructor_id = i.instructor_id
    WHERE i.country = ?
    ORDER BY c.course_id;
    """

    print("Original query results:")
    cursor.execute(original_query, (target_country,))
    original_rows = cursor.fetchall()
    for row in original_rows:
        print(row)

    print("\nApplying indexing suggestions:")
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_instructors_country ON Instructors(country);")
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_courses_instructor ON Courses(instructor_id);")
    conn.commit()
    print("- Index created: idx_instructors_country")
    print("- Index created: idx_courses_instructor")

    print("\nOptimized query results:")
    cursor.execute(optimized_query, (target_country,))
    optimized_rows = cursor.fetchall()
    for row in optimized_rows:
        print(row)

    print("\nVerification (same result set):", original_rows == optimized_rows)

    conn.close()

    print_heading("LAB COMPLETED SUCCESSFULLY")
    print(f"SQLite database file generated: {DB_FILE.resolve()}")


if __name__ == "__main__":
    main()
```

OUTPUT:

```
================================================================================
TASK 5: QUERY OPTIMIZATION
================================================================================
Original query results:
(1, 'SQL Fundamentals', 1)

Applying indexing suggestions:
- Index created: idx_instructors_country
- Index created: idx_courses_instructor

Optimized query results:
(1, 'SQL Fundamentals', 1)

Verification (same result set): True

================================================================================
```

Observation: Optimized JOIN-based query returned the same result as the original subquery version. Suggested indexes improved query design for scalability while preserving correctness (result comparison was True).